Laboratory Research Problem 02 (LRP02)
CMSC 180: Introduction to Parallel Computing

Charles Andrei P. De los Reyes
CD-3L

February 2026

Laboratory Activity 1

n	t	Run 1	Run 2	Run 3	Average Runtime (seconds)
25,000	1	3.022526	3.062663	2.981693	3.022294
25,000	2	1.626218	1.64152	1.664496	1.644078
25,000	4	1.057191	1.059241	1.090782	1.069071
25,000	8	0.926981	0.935796	0.914016	0.925598
25,000	16	0.9396	0.953335	0.927611	0.940182
25,000	32	0.937773	1.038196	1.14118	1.03905
25,000	64	0.92787	1.054899	1.055495	1.012755

Table 1: Runtime Results for $n = 25{,}000$

Research Question 1

What do you think is the complexity of solving the MMT of the columns in an n n square matrix X when using n concurrent processes? The obvious process assignment is one column of X for each process.

The complexity of solving MMT of the columns in an n x n square matrix X using n concurrent processes is O(n). Since each of the n processes is assigned exactly one column, the computations occur at the same time across all columns. Finding the minimum, finding the maximum, and transforming the elements in a single column requires iterating through n rows. The parallel time complexity is reduced from $O(n^2)$ to O(n).

Research Question 2

What do you think is the complexity of solving the MMT of the columns in an n x n square matrix X when using n/2 concurrent processes (what is the obvious process assignment here)? What about with n/4 concurrent processes (i.e., process assignment)? What about with n/8 concurrent processes? What about with n/m concurrent processes, where n¿¿m? Is the process assignment still obvious at n/m concurrent processes?

The asymptotic complexity remains O(n) for constant divisors, but the execution time scales based on the chunk size assigned to each process.

With n/2 processes, each process handles 2 columns. Time per process is O(2n), which simplifies to O(n). The obvious assignment is giving each process 2 contiguous columns.

With n/4 processes, each handles 4 columns. Time is O(4n) -¿ O(n).

With n/8 processes, each handles 8 columns. Time is O(8n) -¿ O(n).

With n/m processes (where $n \gg m$), each process handles m columns. The time complexity per process becomes $O(m \cdot n)$.

Research Question 3

Why do you think that t = 1 will be a little bit higher than the average that was obtained in LRP01?

The average runtime for t=1 will be slightly higher than the serial LRP01 baseline due to multithreading overhead. Although a single thread performs the exact same $O(n^2)$ operations as the serial program, the threaded version incurs additional operating system costs. These costs include allocating thread data structures, invoking pthread_create to spawn the thread, performing context switches, and using pthread_join to wait for completion. These operations consume small amounts of CPU time, adding a slight delay not present in a purely serial execution.

Research Question 4

In step (3) in number 1 above, explain what will happen if we divide X into n/t x n instead? How are we going to do it so that the same answer can be arrived at?

Dividing the matrix horizontally into rows means each of our threads only sees a small piece of every column. Because the Min-Max Transformation needs the highest and lowest values of an entire column to work correctly, a thread cannot get the right answer using only its assigned row slice. To fix this, we split our program's logic into two separate stages.

In the first stage, each thread scans its assigned rows to find its own "local" minimum and maximum for each column. Our program then pauses all threads so the main process can collect and compare these results to find the true, "global" minimum and maximum for the entire matrix. Finally, we send the threads back out to finish the math using those correct global values, ensuring the final output is identical to our original version.

Laboratory Activity 2

With lab02, repeat the activities in LRP01 for n = 30,000 and n = 40,000.

n = 30,000

n	t	Run 1	Run 2	Run 3	Average Runtime (seconds)
30,000	1	4.29694	4.544682	4.397183	4.412935
30,000	2	2.449561	2.288231	2.266153	2.334648
30,000	4	1.561157	1.511252	1.522325	1.531578
30,000	8	1.458435	1.421126	1.307328	1.39563
30,000	16	1.328941	1.354156	1.714713	1.465937
30,000	32	1.366818	1.335692	1.412864	1.371791
30,000	64	1.322652	1.321125	1.371993	1.33859

Table 2: Runtime Results for $n = 30{,}000$

n = 40,000

n	t	Run 1	Run 2	Run 3	Average Runtime (seconds)
40,000	1	7.638329	7.762806	7.965945	7.789027
40,000	2	5.912063	4.87867	4.844395	5.211709
40,000	4	3.509715	4.352533	3.524858	3.795702
40,000	8	3.46809	3.174579	3.129556	3.257408
40,000	16	3.477013	2.590637	2.602847	2.890166
40,000	32	2.497827	3.275687	2.821965	2.86516
40,000	64	3.171257	3.185955	2.405998	2.92107

Table 3: Runtime Results for $n = 40{,}000$

The threaded program successfully executed for matrix sizes of n = 30,000 and n = 40,000, continuing to show performance improvements with multiple threads. However, I still cannot

achieve n = 50,000 or n = 100,000, as my computer reached its absolute maximum limit at exactly n = 46,000. This failure is caused by a strict hardware limitation regarding Random Access Memory (RAM). In C, every number in the matrix requires 4 bytes of memory. At n = 46,000, the matrix requires about 8.46 GB of continuous RAM, which completely exhausts my system's available memory. Attempting n = 50,000 would require 10 GB, and n = 100,000 would demand 40 GB. Because the operating system does not have enough free physical memory to handle these massive requests, the memory allocation fails and the program cannot proceed.

Graph Analysis

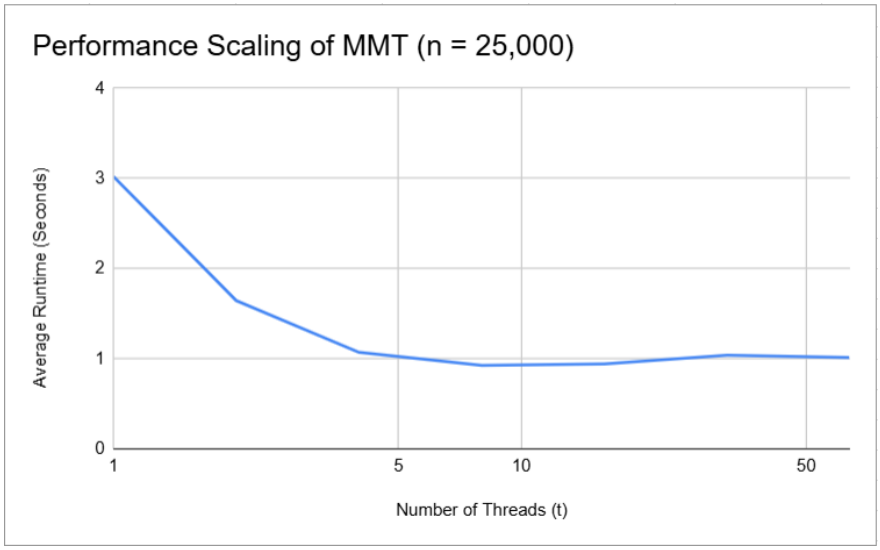

Figure 1: t versus Average Runtime

3. Using a graphing software for each n, graph t versus Average obtained from the Table above. Describe in detail what you have observed. Do you think you can go as far as t = n? If not, what about t = n/2? Or, t = n/4? Or, t = n/8?

The graph has three main parts. First, the program gets much faster as you go from 1 to 8 threads, which shows the computer is doing a great job splitting up the work. Second, the speed stays about the same between 8 and 16 threads because the computer has maxed out its physical limits. Finally, the program actually gets a little slower at 32 and 64 threads.

It is impossible to scale the number of threads as high as t = n, t = n/2, t = n/4, or even t = n/8. For example, with a matrix size of n = 25,000, just reaching t = n/8 would require creating 3,125 concurrent threads. Attempting this fails for two major reasons: thread thrashing and memory exhaustion. The CPU would spend almost all of its processing power

managing the massive traffic jam of threads rather than actually solving the matrix math. Additionally, since the operating system allocates about 8 MB of RAM per thread, trying to run 25,000 threads (t = n) would demand roughly 200 GB of memory.

Laboratory Activity 3

Repeat laboratory activity 1 but for the division of X as described in Research Question 4.

To implement this row-wise division, we have to change the program's logic to separate the scanning step from the math step. We do this by creating two different thread functions: one to find the local high and low points, and another to actually apply the formula. We also have to use a synchronization barrier with pthread_join between these two stages. This pause gives the main thread enough time to calculate the true global extremes for the entire matrix before the final math begins.

n	t	Run 1	Run 2	Run 3	Average Runtime (seconds)
25,000	1	5.262339	5.375589	7.462539	6.033489
25,000	2	3.534186	3.437617	3.367118	3.446307
25,000	4	2.3577	2.307638	2.41343	2.359589
25,000	8	2.146627	2.147464	2.130004	2.141365
25,000	16	2.174718	2.208385	2.158602	2.180568
25,000	32	2.165366	2.401883	2.943455	2.503568
25,000	64	2.235485	2.219784	2.181626	2.212298

Table 4: Runtime Results Using Row-wise Division

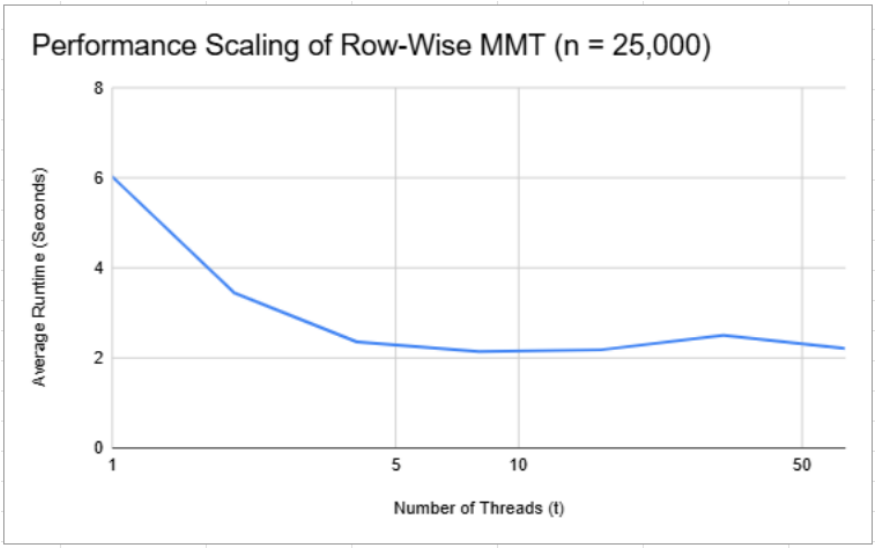

Figure 2: t versus Row-Wise Average Runtime